Linguagens Formais e Compiladores

Aula 1: o que acontece quando você aperta *build*

Lucas Vinicius Oliveira Cardoso

Ciência da Computação | 2026/2

Você já compilou milhares de programas.

Esta disciplina é sobre os poucos milissegundos entre o *build* e a mensagem de erro.

1
Onde isto aparece

Compiladores não são só para linguagens de programação

Compiladores não são só para linguagens de programação

- Um banco de dados traduz SQL em plano de execução

Compiladores não são só para linguagens de programação

- Um banco de dados traduz SQL em plano de execução
- Um navegador traduz CSS em regras de layout

Compiladores não são só para linguagens de programação

- Um banco de dados traduz SQL em plano de execução
- Um navegador traduz CSS em regras de layout
- Uma planilha traduz `=SOMA(A1:A9)` em operações

Compiladores não são só para linguagens de programação

- Um banco de dados traduz SQL em plano de execução
- Um navegador traduz CSS em regras de layout
- Uma planilha traduz `=SOMA(A1:A9)` em operações
- Uma placa de vídeo compila *shaders*

Compiladores não são só para linguagens de programação

- Um banco de dados traduz SQL em plano de execução
- Um navegador traduz CSS em regras de layout
- Uma planilha traduz `=SOMA(A1:A9)` em operações
- Uma placa de vídeo compila *shaders*
- Um editor traduz Markdown em HTML

Compiladores não são só para linguagens de programação

- Um banco de dados traduz SQL em plano de execução
- Um navegador traduz CSS em regras de layout
- Uma planilha traduz `=SOMA(A1:A9)` em operações
- Uma placa de vídeo compila *shaders*
- Um editor traduz Markdown em HTML
- Toda expressão regular é compilada antes de rodar

Compiladores não são só para linguagens de programação

- Um banco de dados traduz SQL em plano de execução
- Um navegador traduz CSS em regras de layout
- Uma planilha traduz `=SOMA(A1:A9)` em operações
- Uma placa de vídeo compila *shaders*
- Um editor traduz Markdown em HTML
- Toda expressão regular é compilada antes de rodar

Você vai escrever tradutores mesmo que nunca trabalhe com linguagens.

Por que esta disciplina existe

- Antes eram duas: Linguagens Formais e Compiladores.

Por que esta disciplina existe

- Antes eram duas: Linguagens Formais e Compiladores.
- Você estudava autômatos num semestre e construía um analisador léxico no seguinte.

Por que esta disciplina existe

- Antes eram duas: Linguagens Formais e Compiladores.
- Você estudava autômatos num semestre e construía um analisador léxico no seguinte.
- Sem que ninguém dissesse que a segunda coisa é a primeira, escrita em código.

Por que esta disciplina existe

- Antes eram duas: Linguagens Formais e Compiladores.
- Você estudava autômatos num semestre e construía um analisador léxico no seguinte.
- Sem que ninguém dissesse que a segunda coisa é a primeira, escrita em código.

O QUE MUDA AQUI

Cada formalismo aparece imediatamente antes da fase que o consome. Autômatos antes do léxico, gramáticas antes do sintático.

2
Um programa, quatro defeitos

O programa que funciona

```c
int main(void) {
    int x = 10;
    int y = x * 2;
    printf("%d\n", y);
    return 0;
}
```

```
$ gcc -c 0-correto.c
```

nenhuma saída, compilou

O programa que funciona

```c
int main(void) {
    int x = 10;
    int y = x * 2;
    printf("%d\n", y);
    return 0;
}
```

```
$ gcc -c 0-correto.c
```

nenhuma saída, compilou

Vamos quebrá-lo de quatro formas. Tente prever cada mensagem.

Defeito 1

```
int main(void) {
    int x = 1 @ 0;
    int y = x * 2;
    return y;
}
```

Defeito 1

```
int main(void) {
    int x = 1@0;
    int y = x * 2;
    return y;
}
```

```
1-lexico.c:2:14: error: stray '@' in program
    2 | int x = 1@0;
      |          ^
```

Defeito 1

```
int main(void) {
    int x = 1@0;
    int y = x * 2;
    return y;
}
```

```
1-lexico.c:2:14: error: stray '@' in program
    2 | int x = 1@0;
      |          ^
```

Bastou olhar um símbolo. Análise léxica.

Defeito 2

```
int main(void) {
    int x = 10;
    int y = ( x * 2;
    return y;
}
```

Defeito 2

```c
int main(void) {
    int x = 10;
    int y = ( x * 2;
    return y;
}
```

```
2-sintatico.c:3:19: error: expected ')' before ';' token
    3 | int y = (x * 2;
      |          ~ ^
      |            )
```

Defeito 2

```c
int main(void) {
    int x = 10;
    int y = ( x * 2;
    return y;
}
```

```
2-sintatico.c:3:19: error: expected ')' before ';' token
    3 | int y = (x * 2;
      |          ~ ^
      |            )
```

Todos os símbolos valem. O problema é a ordem.
Análise sintática.

Defeito 3

```
int main(void) {
    int x = 10;
    int y = z * 2;
    return y;
}
```

Defeito 3

```c
int main(void) {
    int x = 10;
    int y = z * 2;
    return y;
}
```

```
3-semantico-nome.c:3:13: error: 'z' undeclared (first use in this function)
    3 | int y = z * 2;
      |         ^
```

Defeito 3

```c
int main(void) {
    int x = 10;
    int y = z * 2;
    return y;
}
```

```
3-semantico-nome.c:3:13: error: 'z' undeclared (first use in this function)
    3 | int y = z * 2;
      |         ^
```

A forma está perfeita. Faltou memória do que foi declarado.

Defeito 4

```
int main(void) {
    int x = 10;
    int y = x * "dois";
    return y;
}
```

Defeito 4

```c
int main(void) {
    int x = 10;
    int y = x * "dois";
    return y;
}
```

```
4-semantico-tipo.c:3:15: error: invalid operands to binary * (have 'int' and 'char *')
    3 | int y = x * "dois";
      |             ^
      |             char *
```

Defeito 4

```c
int main(void) {
    int x = 10;
    int y = x * "dois";
    return y;
}
```

```
4-semantico-tipo.c:3:15: error: invalid operands to binary * (have 'int' and 'char *')
    3 | int y = x * "dois";
      |             ^
      |             char *
```

Nomes declarados, forma correta. Faltou saber os tipos.

Dois defeitos, um arquivo

```c
int main(void) {
    int x = 10;
    int y = (x * 2; /* sintatico */
    int w = x * "dois"; /* semantico */
    return 0;
}
```

Dois defeitos, um arquivo

```
int main(void) {
    int x = 10;
    int y = (x * 2; /* sintatico */
    int w = x * "dois"; /* semantico */
    return 0;
}
```

O gcc vai reclamar da linha 4?

A resposta

```
5-dois-defeitos.c:3:19: error: expected ')' before ';' token
5-dois-defeitos.c:6:1: error: expected ',' or ';' before '}' token
5-dois-defeitos.c:6:1: error: expected declaration or statement at end of input
```

A resposta

```
5-dois-defeitos.c:3:19: error: expected ')' before ';' token
5-dois-defeitos.c:6:1: error: expected ',' or ';' before '}' token
5-dois-defeitos.c:6:1: error: expected declaration or statement at end of input
```

- Três erros relatados, e todos os três são sintáticos.
- O primeiro é real. Os outros dois são cascata da recuperação.
- A linha 4 não é mencionada.

Agora consertamos só o parêntese

```
int main(void) {
    int x = 10;
    int y = (x * 2 ) ;
    int w = x * "dois"; /* intocado */
    return 0;
}
```

Agora consertamos só o parêntese

```c
int main(void) {
    int x = 10;
    int y = (x * 2 );
    int w = x * "dois"; /* intocado */
    return 0;
}
```

```
6-parentese-consertado.c:4:15: error: invalid operands to binary *
    4 | int w = x * "dois"; (have 'int' and 'char *')
      |             ^
```

Agora consertamos só o parêntese

```c
int main(void) {
    int x = 10;
    int y = (x * 2 );
    int w = x * "dois"; /* intocado */
    return 0;
}
```

```
6-parentese-consertado.c:4:15: error: invalid operands to binary *
    4 | int w = x * "dois"; (have 'int' and 'char *')
      |             ^
```

O erro de tipo apareceu. Estava lá o tempo todo.

Por que o erro de tipo estava invisível

1. O compilador nunca chegou à análise semântica.

Por que o erro de tipo estava invisível

1. O compilador nunca chegou à análise semântica.
2. A análise sintática falhou antes, e ela vem primeiro.

Por que o erro de tipo estava invisível

1. O compilador nunca chegou à análise semântica.
2. A análise sintática falhou antes, e ela vem primeiro.
3. Sem árvore construída, não há o que anotar com tipos.

Por que o erro de tipo estava invisível

1. O compilador nunca chegou à análise semântica.
2. A análise sintática falhou antes, e ela vem primeiro.
3. Sem árvore construída, não há o que anotar com tipos.
4. Removido o obstáculo sintático, a fase seguinte rodou e acusou.

Por que o erro de tipo estava invisível

1. O compilador nunca chegou à análise semântica.
2. A análise sintática falhou antes, e ela vem primeiro.
3. Sem árvore construída, não há o que anotar com tipos.
4. Removido o obstáculo sintático, a fase seguinte rodou e acusou.

A CONCLUSÃO QUE VALE O SEMESTRE

As fases não são organização didática de livro. São **dependência real**: cada uma consome o que a anterior produziu. Vocês acabaram de ver o compilador provando isso, nos dois sentidos.

Um detalhe que economiza horas

- O defeito 1 gerou dois erros, não um.

Um detalhe que economiza horas

- O defeito 1 gerou dois erros, não um.
- O segundo é fantasma, efeito da tentativa de recuperação.

Um detalhe que economiza horas

- O defeito 1 gerou dois erros, não um.
- O segundo é fantasma, efeito da tentativa de recuperação.
- O compilador tentou seguir adiante e se perdeu.

Um detalhe que economiza horas

- O defeito 1 gerou dois erros, não um.
- O segundo é fantasma, efeito da tentativa de recuperação.
- O compilador tentou seguir adiante e se perdeu.

O primeiro erro é sempre o real. Corrija-o e recompile.

O que acabamos de descobrir

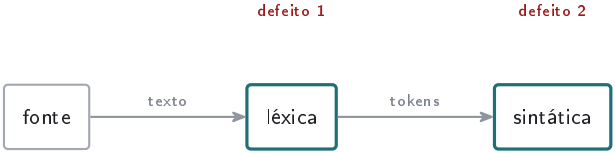
fonte

O que acabamos de descobrir

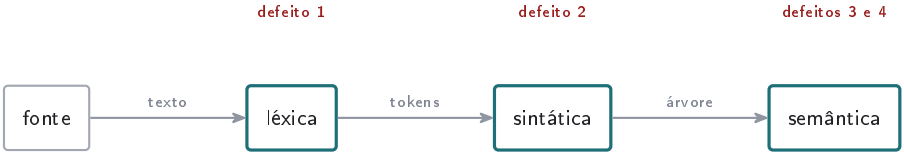
defeito 1



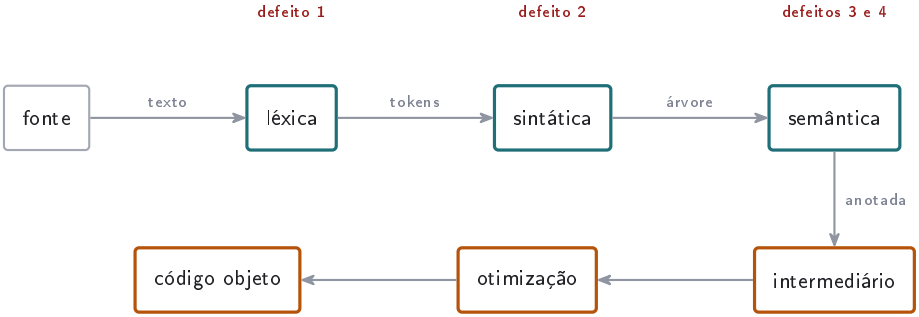
O que acabamos de descobrir



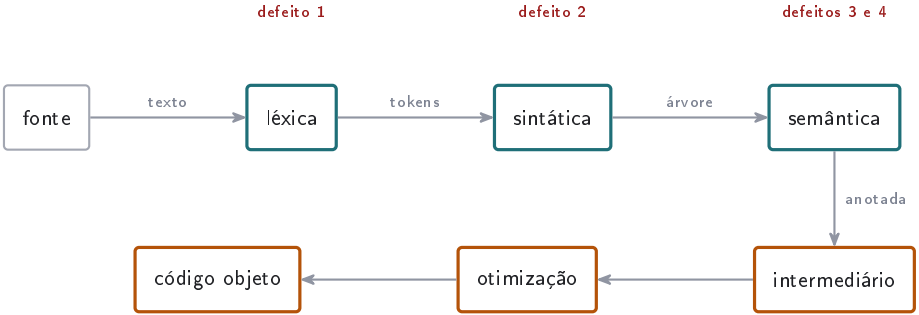
O que acabamos de descobrir



O que acabamos de descobrir



O que acabamos de descobrir



ATRAVESSANDO TODAS AS FASES

A **tabela de símbolos**, que registra o que cada nome significa, e o **atendimento a erros**, que funciona em qualquer ponto.

3
As palavras que vamos usar

Sete palavras, o semestre todo

Antes de continuar, precisamos combinar o significado de sete palavras. Elas vão reaparecer em toda aula deste curso, e usá-las de forma imprecisa é a maior fonte de confusão no assunto.

símbolo	alfabeto	palavra
linguagem	gramática	
token	lexema	padrão

Sete palavras, o semestre todo

Antes de continuar, precisamos combinar o significado de sete palavras. Elas vão reaparecer em toda aula deste curso, e usá-las de forma imprecisa é a maior fonte de confusão no assunto.

símbolo	alfabeto	palavra
linguagem	gramática	
token	lexema	padrão

As cinco primeiras vêm da matemática. As três últimas vêm da engenharia de compiladores. Elas se encontram.

Símbolo, alfabeto, palavra

- **Símbolo** é um elemento indivisível. a, 7, ;

Símbolo, alfabeto, palavra

- **Símbolo** é um elemento indivisível. a, 7, ;
- **Alfabeto** é um conjunto finito de símbolos. Escrevemos Σ.

  $\Sigma = \{\mathtt{a}, \mathtt{b}\}$ ou $\Sigma =$ todos os caracteres do teclado

Símbolo, alfabeto, palavra

- **Símbolo** é um elemento indivisível. a, 7, ;
- **Alfabeto** é um conjunto finito de símbolos. Escrevemos Σ.
  $\Sigma = \{\mathtt{a}, \mathtt{b}\}$ ou $\Sigma =$ todos os caracteres do teclado
- **Palavra** é uma sequência finita de símbolos. `total`, `12.5`, ;

Símbolo, alfabeto, palavra

- **Símbolo** é um elemento indivisível. `a`, `7`, `;`
- **Alfabeto** é um conjunto finito de símbolos. Escrevemos Σ.
  $\Sigma = \{\texttt{a}, \texttt{b}\}$ ou $\Sigma =$ todos os caracteres do teclado
- **Palavra** é uma sequência finita de símbolos. `total`, `12.5`, `;`

A PALAVRA VAZIA

Existe uma palavra com zero símbolos. Ela se chama **épsilon**, escreve-se ε, e não é o mesmo que "nada": é uma palavra legítima, de comprimento zero. Vamos precisar dela.

Símbolo, alfabeto, palavra

- **Símbolo** é um elemento indivisível. `a`, `7`, `;`
- **Alfabeto** é um conjunto finito de símbolos. Escrevemos Σ.
  $\Sigma = \{\texttt{a}, \texttt{b}\}$ ou $\Sigma =$ todos os caracteres do teclado
- **Palavra** é uma sequência finita de símbolos. `total`, `12.5`, `;`

A PALAVRA VAZIA

Existe uma palavra com zero símbolos. Ela se chama **épsilon**, escreve-se ε, e não é o mesmo que "nada": é uma palavra legítima, de comprimento zero. Vamos precisar dela.

"Palavra"aqui não quer dizer palavra de idioma. `x9_z` é uma palavra.

Linguagem

Uma linguagem é um *conjunto de palavras*.

Linguagem

Uma linguagem é um *conjunto de palavras*.

- O conjunto de todos os identificadores válidos é uma linguagem.

Linguagem

Uma linguagem é um *conjunto de palavras*.

- O conjunto de todos os identificadores válidos é uma linguagem.
- O conjunto de todos os números reais bem escritos é uma linguagem.

Linguagem

Uma linguagem é um *conjunto de palavras*.

- O conjunto de todos os identificadores válidos é uma linguagem.
- O conjunto de todos os números reais bem escritos é uma linguagem.
- O conjunto de todos os programas C que compilam é uma linguagem.

Linguagem

Uma linguagem é um *conjunto de palavras*.

- O conjunto de todos os identificadores válidos é uma linguagem.
- O conjunto de todos os números reais bem escritos é uma linguagem.
- O conjunto de todos os programas C que compilam é uma linguagem.

O QUE ISSO DESLOCA

"Linguagem"deixa de ser algo vago, com sintaxe e cultura e bibliotecas, e passa a ser um objeto matemático: um conjunto. Pode ser finito ou infinito. Reconhecer uma linguagem é decidir se uma palavra pertence ao conjunto.

Gramática

Uma gramática é um conjunto de *regras que geram* palavras.

Gramática

Uma gramática é um conjunto de *regras que geram* palavras.

Uma regra diz como substituir um nome por outra coisa:

$$\texttt{comando} \rightarrow \texttt{se condição comando}$$
$$\texttt{comando} \rightarrow \texttt{variável = expressão}$$

Gramática

Uma gramática é um conjunto de *regras que geram* palavras.

Uma regra diz como substituir um nome por outra coisa:

$$\texttt{comando} \rightarrow \texttt{se condição comando}$$
$$\texttt{comando} \rightarrow \texttt{variável = expressão}$$

A DIVISÃO DE TRABALHO

O **autômato** reconhece: recebe uma palavra e responde sim ou não. A **gramática** gera: produz palavras aplicando regras. Duas formas de descrever o mesmo conjunto, por dentro e por fora.

Token, lexema, padrão

As três palavras do lado da engenharia. Voltando a `total = 12.5;`

Padrão	a regra	"letra, seguida de letras ou dígitos"
Lexema	o pedaço concreto	`total`
Token	a categoria	ID, mais os atributos

Token, lexema, padrão

As três palavras do lado da engenharia. Voltando a `total = 12.5;`

Padrão	a regra	"letra, seguida de letras ou dígitos"
Lexema	o pedaço concreto	`total`
Token	a categoria	ID, mais os atributos

A RELAÇÃO ENTRE OS DOIS LADOS

Um **padrão** descreve uma **linguagem**: o conjunto de todos os lexemas que ele aceita. Reconhecer um token é decidir se um pedaço do texto pertence a essa linguagem. É exatamente o problema que o autômato resolve.

Amarrando

`t`, `o`, `=`	são **símbolos**
o teclado inteiro	é o **alfabeto**
`total`	é uma **palavra**, e também um **lexema**
todos os identificadores	formam uma **linguagem**
"letra seguida de letras ou dígitos"	é o **padrão** que a descreve
ID	é o **token** produzido

Amarrando

`t`, `o`, `=`	são **símbolos**
o teclado inteiro	é o **alfabeto**
`total`	é uma **palavra**, e também um **lexema**
todos os identificadores	formam uma **linguagem**
"letra seguida de letras ou dígitos"	é o **padrão** que a descreve
ID	é o **token** produzido

Agora podemos falar de autômatos sem ambiguidade.

4
Autômatos finitos

Como um programa reconhece um identificador?

- Você poderia escrever uma sequência de `if` aninhados.

Como um programa reconhece um identificador?

- Você poderia escrever uma sequência de `if` aninhados.
- Funciona, mas não escala: cada categoria de token viraria outro emaranhado.

Como um programa reconhece um identificador?

- Você poderia escrever uma sequência de `if` aninhados.
- Funciona, mas não escala: cada categoria de token viraria outro emaranhado.
- Existe uma máquina mais simples que resolve o problema inteiro, e ela é a base de todo analisador léxico.

Como um programa reconhece um identificador?

- Você poderia escrever uma sequência de `if` aninhados.
- Funciona, mas não escala: cada categoria de token viraria outro emaranhado.
- Existe uma máquina mais simples que resolve o problema inteiro, e ela é a base de todo analisador léxico.

Autômato finito determinístico

O vocabulário dos desenhos

Antes de qualquer autômato, os quatro tipos de círculo:



estado comum
passagem

inicial
flecha vinda
do nada

de aceitação
círculo duplo

inicial e de aceitação
os dois ao
mesmo tempo

O vocabulário dos desenhos

Antes de qualquer autômato, os quatro tipos de círculo:



estado comum
passagem

inicial
flecha vinda
do nada

de aceitação
círculo duplo

inicial e de aceitação
os dois ao
mesmo tempo

E AS FLECHAS

Cada flecha carrega o **símbolo lido** para percorrê-la. Uma flecha que volta ao mesmo estado é um laço, e significa "continue lendo isto sem mudar de situação".

Como a máquina funciona

1. Comece no estado inicial.

Como a máquina funciona

1. Comece no estado inicial.
2. Leia o próximo símbolo e siga a flecha correspondente.

Como a máquina funciona

1. Comece no estado inicial.
2. Leia o próximo símbolo e siga a flecha correspondente.
3. Se não existir flecha para aquele símbolo, rejeite.

Como a máquina funciona

1. Comece no estado inicial.
2. Leia o próximo símbolo e siga a flecha correspondente.
3. Se não existir flecha para aquele símbolo, rejeite.
4. Acabou a palavra: se estiver num estado de aceitação, aceite.

Como a máquina funciona

1. Comece no estado inicial.
2. Leia o próximo símbolo e siga a flecha correspondente.
3. Se não existir flecha para aquele símbolo, rejeite.
4. Acabou a palavra: se estiver num estado de aceitação, aceite.

O MAL-ENTENDIDO MAIS COMUM

Círculo duplo **não** significa "pare aqui". Significa "se a palavra terminou aqui, aceite". A máquina pode sair de um estado de aceitação, passar por estados comuns e voltar, quantas vezes quiser.

O estado é tudo o que a máquina lembra.

O que a máquina lembra

O estado é tudo o que a máquina lembra.

- Ela não sabe quantos símbolos já leu.

O estado é tudo o que a máquina lembra.

- Ela não sabe quantos símbolos já leu.
- Não sabe quais eram.

O estado é tudo o que a máquina lembra.

- Ela não sabe quantos símbolos já leu.
- Não sabe quais eram.
- Não pode voltar atrás para conferir.

O estado é tudo o que a máquina lembra.

- Ela não sabe quantos símbolos já leu.
- Não sabe quais eram.
- Não pode voltar atrás para conferir.

GUARDE ESTA FRASE

Ela vai virar uma demonstração de impossibilidade, e vai explicar por que um analisador léxico não consegue verificar parênteses balanceados.

A definição formal

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

A definição formal

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto** de entrada, finito

A definição formal

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto** de entrada, finito
- Q conjunto **finito** de estados

A definição formal

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto** de entrada, finito
- Q conjunto **finito** de estados
- $\delta : Q \times \Sigma \to Q$ a **função de transição**

A definição formal

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto** de entrada, finito
- Q conjunto **finito** de estados
- $\delta : Q \times \Sigma \to Q$ a **função de transição**
- $q_0 \in Q$ o **estado inicial**

A definição formal

Um autômato finito determinístico é uma quíntupla

$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto** de entrada, finito
- Q conjunto **finito** de estados
- $\delta : Q \times \Sigma \to Q$ a **função de transição**
- $q_0 \in Q$ o **estado inicial**
- $F \subseteq Q$ os estados **finais**

A definição formal

Um autômato finito determinístico é uma quíntupla

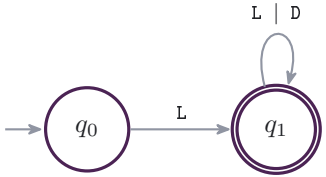
$$M = (\Sigma,\ Q,\ \delta,\ q_0,\ F)$$

- Σ o **alfabeto** de entrada, finito
- Q conjunto **finito** de estados
- $\delta : Q \times \Sigma \to Q$ a **função de transição**
- $q_0 \in Q$ o **estado inicial**
- $F \subseteq Q$ os estados **finais**

ONDE ESTÁ A LIMITAÇÃO

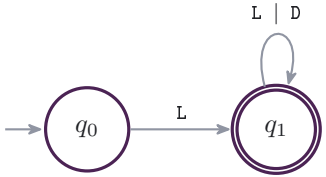
Q é finito. Esse detalhe é a razão de tudo o que esta máquina consegue e não consegue fazer, e voltaremos a ele.

A definição, aplicada



L letra, D dígito

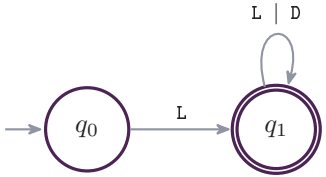
A definição, aplicada



- $\Sigma = \{\texttt{L}, \texttt{D}\}$

L letra, D dígito

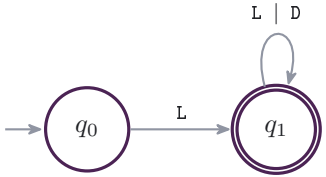
A definição, aplicada



- $\Sigma = \{\texttt{L}, \texttt{D}\}$
- $Q = \{q_0, q_1\}$

L letra, D dígito

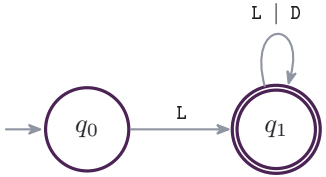
A definição, aplicada



- $\Sigma = \{\texttt{L}, \texttt{D}\}$
- $Q = \{q_0, q_1\}$
- $\delta(q_0, \texttt{L}) = q_1, \;\; \delta(q_1, \texttt{L}) = q_1, \;\; \delta(q_1, \texttt{D}) = q_1$

L letra, D dígito

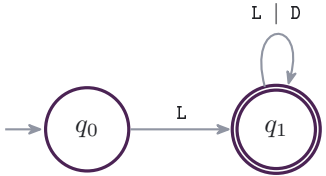
A definição, aplicada



- $\Sigma = \{\mathtt{L}, \mathtt{D}\}$
- $Q = \{q_0, q_1\}$
- $\delta(q_0, \mathtt{L}) = q_1, \;\; \delta(q_1, \mathtt{L}) = q_1, \;\; \delta(q_1, \mathtt{D}) = q_1$
- q_0 é o estado inicial

L letra, D dígito

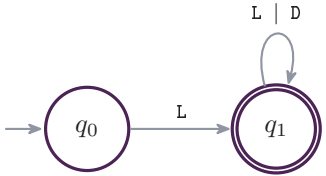
A definição, aplicada



L letra, D dígito

- $\Sigma = \{\mathtt{L}, \mathtt{D}\}$
- $Q = \{q_0, q_1\}$
- $\delta(q_0, \mathtt{L}) = q_1, \;\; \delta(q_1, \mathtt{L}) = q_1, \;\; \delta(q_1, \mathtt{D}) = q_1$
- q_0 é o estado inicial
- $F = \{q_1\}$

A definição, aplicada



L letra, D dígito

- $\Sigma = \{\mathtt{L}, \mathtt{D}\}$
- $Q = \{q_0, q_1\}$
- $\delta(q_0, \mathtt{L}) = q_1, \;\; \delta(q_1, \mathtt{L}) = q_1, \;\; \delta(q_1, \mathtt{D}) = q_1$
- q_0 é o estado inicial
- $F = \{q_1\}$

E $\delta(q_0, \mathtt{D})$?

Não está definida, e é exatamente isso que rejeita 9x como identificador. Uma função de transição incompleta significa que a ausência de flecha é rejeição.

Quando a máquina aceita

Já sabemos o que δ faz com um símbolo. E com uma palavra inteira?

Quando a máquina aceita

Já sabemos o que δ faz com um símbolo. E com uma palavra inteira?

1. Comece em q_0.

Quando a máquina aceita

Já sabemos o que δ faz com um símbolo. E com uma palavra inteira?

1. Comece em q_0.
2. Aplique δ ao primeiro símbolo. Depois ao segundo. E assim por diante.

Quando a máquina aceita

Já sabemos o que δ faz com um símbolo. E com uma palavra inteira?

1. Comece em q_0.
2. Aplique δ ao primeiro símbolo. Depois ao segundo. E assim por diante.
3. Quando a palavra acabar, olhe o estado em que você parou.

Quando a máquina aceita

Já sabemos o que δ faz com um símbolo. E com uma palavra inteira?

1. Comece em q_0.
2. Aplique δ ao primeiro símbolo. Depois ao segundo. E assim por diante.
3. Quando a palavra acabar, olhe o estado em que você parou.

M aceita a palavra se esse estado final pertence a F.

Quando a máquina aceita

Já sabemos o que δ faz com um símbolo. E com uma palavra inteira?

1. Comece em q_0.
2. Aplique δ ao primeiro símbolo. Depois ao segundo. E assim por diante.
3. Quando a palavra acabar, olhe o estado em que você parou.

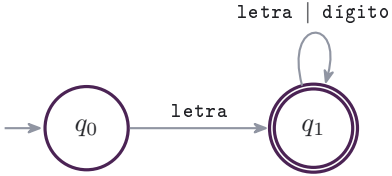
M aceita a palavra se esse estado final pertence a F.

A LINGUAGEM RECONHECIDA

Juntando todas as palavras que M aceita, obtemos a **linguagem reconhecida** por M, escrita $L(M)$. Perceba que um autômato não aceita apenas uma palavra: ele define um conjunto inteiro.

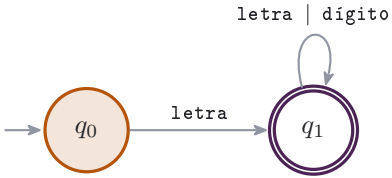
Um autômato para identificadores

Um identificador começa com letra e segue com letras ou dígitos.



Um autômato para identificadores

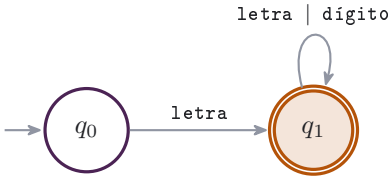
Um identificador começa com letra e segue com letras ou dígitos.



`total`

Um autômato para identificadores

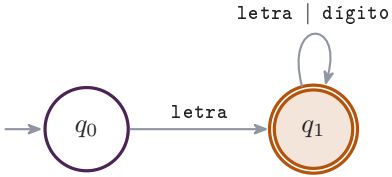
Um identificador começa com letra e segue com letras ou dígitos.



`total`

Um autômato para identificadores

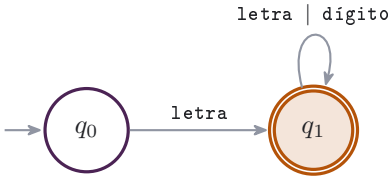
Um identificador começa com letra e segue com letras ou dígitos.



`total`

Um autômato para identificadores

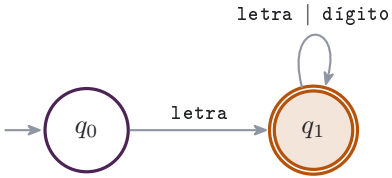
Um identificador começa com letra e segue com letras ou dígitos.



`total`

Um autômato para identificadores

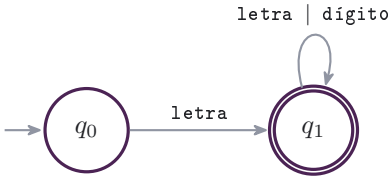
Um identificador começa com letra e segue com letras ou dígitos.



`total`

Um autômato para identificadores

Um identificador começa com letra e segue com letras ou dígitos.



`total` aceita

A mesma máquina, como tabela

δ	L	D
$\rightarrow q_0$	q_1	—
$* q_1$	q_1	q_1

$\rightarrow$ inicial $*$ final

A mesma máquina, como tabela

δ	L	D
$\rightarrow q_0$	q_1	—
$* q_1$	q_1	q_1

$\rightarrow$ inicial $*$ final

- A tabela contém a mesma informação do desenho.
- Ler um símbolo é uma consulta indexada, sem busca.
- Célula vazia significa rejeitar.

A mesma máquina, como tabela

δ	L	D
$\rightarrow q_0$	q_1	—
$*\,q_1$	q_1	q_1

$\rightarrow$ inicial $*$ final

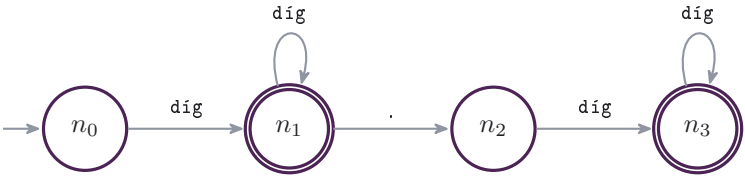
- A tabela contém a mesma informação do desenho.
- Ler um símbolo é uma consulta indexada, sem busca.
- Célula vazia significa rejeitar.

A PONTE PARA O CÓDIGO

Esta tabela não é notação alternativa: é a **estrutura de dados** do analisador léxico. Um scanner escrito à mão é esta tabela mais um laço de três linhas, e é por isso que reconhecer tokens custa tempo linear e memória constante.

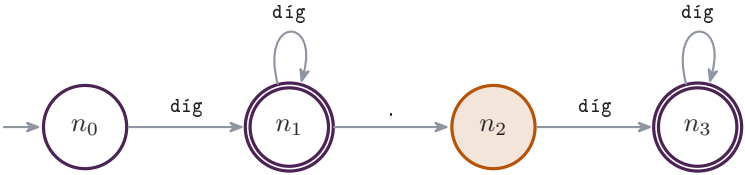
Um autômato para números

Agora um caso com armadilha: `12.5` é número, mas `12.` não deveria ser.

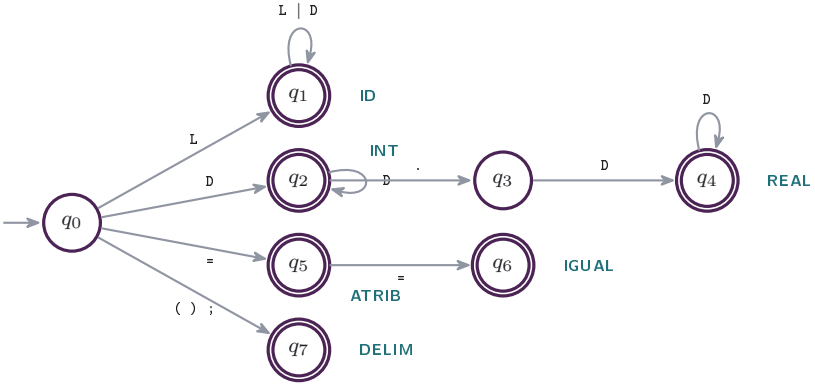

dígdígdígd

Um autômato para números

Agora um caso com armadilha: `12.5` é número, mas `12.` não deveria ser.


dígdígdígd

Um autômato de verdade



Cinco categorias de token. `L` letra, `D` dígito.

O que esse desenho diz

A VIRADA DE CHAVE

Não é um autômato por categoria. É **um só**, e o estado de aceitação onde a máquina para diz **qual** token foi reconhecido.

O que esse desenho diz

A VIRADA DE CHAVE

Não é um autômato por categoria. É **um só**, e o estado de aceitação onde a máquina para diz **qual** token foi reconhecido.

- q_3 **não é de aceitação**, e é isso que rejeita 12.

O que esse desenho diz

A VIRADA DE CHAVE

Não é um autômato por categoria. É **um só**, e o estado de aceitação onde a máquina para diz **qual** token foi reconhecido.

- q_3 **não é de aceitação**, e é isso que rejeita 12.
- q_5 **e** q_6 **são ambos de aceitação:** = para em q_5 e dá ATRIB; == segue a q_6 e dá IGUAL

O que esse desenho diz

A VIRADA DE CHAVE

Não é um autômato por categoria. É **um só**, e o estado de aceitação onde a máquina para diz **qual** token foi reconhecido.

- q_3 **não é de aceitação**, e é isso que rejeita `12`.
- q_5 **e** q_6 **são ambos de aceitação:** = para em q_5 e dá ATRIB; == segue a q_6 e dá IGUAL
- **não há flecha de** q_0 **com dígito**, e por isso `9x` não é identificador

O que esse desenho diz

A VIRADA DE CHAVE

Não é um autômato por categoria. É **um só**, e o estado de aceitação onde a máquina para diz **qual** token foi reconhecido.

- q_3 **não é de aceitação**, e é isso que rejeita 12.
- q_5 **e** q_6 **são ambos de aceitação:** = para em q_5 e dá ATRIB; == segue a q_6 e dá IGUAL
- **não há flecha de** q_0 **com dígito**, e por isso 9x não é identificador

O segundo item é o casamento mais longo, desenhado.

Traçando uma linha de código

```
total = 12.5;
```

Traçando uma linha de código

```
total = 12.5;
```

$q_0 \rightarrow q_1$ `total` ID

Traçando uma linha de código

```
total = 12.5;
```

$q_0 \rightarrow q_1$	`total`	ID
$q_0 \rightarrow q_5$	`=`	ATRIB

Traçando uma linha de código

```
total = 12.5;
```

$q_0 \rightarrow q_1$	`total`	ID
$q_0 \rightarrow q_5$	`=`	ATRIB
$q_0 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4$	`12.5`	REAL

Traçando uma linha de código

```
total = 12.5;
```

$q_0 \rightarrow q_1$	`total`	ID
$q_0 \rightarrow q_5$	`=`	ATRIB
$q_0 \rightarrow q_2 \rightarrow q_3 \rightarrow q_4$	`12.5`	REAL
$q_0 \rightarrow q_7$	`;`	DELIM

Traçando uma linha de código

```
total = 12.5;
```

$q_0 \to q_1$	`total`	ID
$q_0 \to q_5$	`=`	ATRIB
$q_0 \to q_2 \to q_3 \to q_4$	`12.5`	REAL
$q_0 \to q_7$	`;`	DELIM

O QUE FAZ ISSO FUNCIONAR

A máquina roda **uma vez por token** e volta a q_0 no início de cada um. O analisador léxico é este autômato dentro de um laço.

De volta ao começo da aula



De volta ao começo da aula



A CAIXA DEIXOU DE SER CAIXA-PRETA

Na primeira metade da aula, "análise léxica"era um retângulo num diagrama. Agora vocês sabem o que tem dentro: um autômato finito e um laço.

Por que isto importa hoje

- Estes dois desenhos são o analisador léxico.

Por que isto importa hoje

- Estes dois desenhos são o analisador léxico.
- Você vai transformá-los em código, à mão.

Por que isto importa hoje

- Estes dois desenhos são o analisador léxico.
- Você vai transformá-los em código, à mão.
- Depois vai deixar uma ferramenta gerar o código a partir deles.

Por que isto importa hoje

- Estes dois desenhos são o analisador léxico.
- Você vai transformá-los em código, à mão.
- Depois vai deixar uma ferramenta gerar o código a partir deles.

Desenhar autômatos é a habilidade central da primeira metade do curso.

5

Três ideias que valem a aula

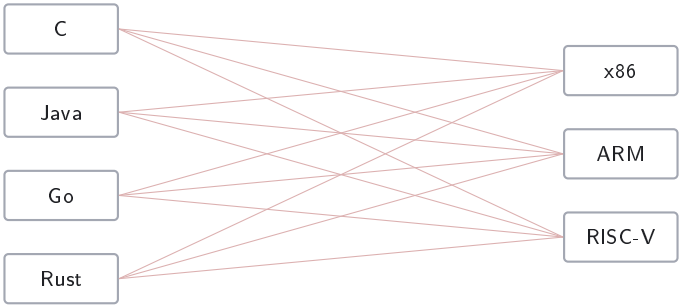
Por que separar em fases

Suponha 4 linguagens e 3 arquiteturas de máquina.



Por que separar em fases

Suponha 4 linguagens e 3 arquiteturas de máquina.



12 compiladores para escrever

Por que separar em fases

Suponha 4 linguagens e 3 arquiteturas de máquina.



4 front-ends mais 3 back-ends: 7 peças

Por que separar em fases

Suponha 4 linguagens e 3 arquiteturas de máquina.



4 front-ends mais 3 back-ends: 7 peças

$N \times M$ **vira** $N + M$**. Por isso o LLVM existe.**

Mensagens de erro são o produto

O mesmo defeito, relatado por dois compiladores diferentes:

UM COMPILADOR RUIM DIRIA

```
erro de sintaxe
```

Tecnicamente correto. Você ainda não sabe *onde*, nem *o que* faltou.

Mensagens de erro são o produto

O mesmo defeito, relatado por dois compiladores diferentes:

UM COMPILADOR RUIM DIRIA

```
erro de sintaxe
```

Tecnicamente correto. Você ainda não sabe *onde*, nem *o que* faltou.

O QUE O GCC DIZ

```
error: expected ')' before ';'
    5 | int y = (x * 2;
      |        ~      ^
      |                )
```

Diz a linha, a coluna, o que esperava, e sugere a correção.

Mensagens de erro são o produto

O mesmo defeito, relatado por dois compiladores diferentes:

UM COMPILADOR RUIM DIRIA

```
erro de sintaxe
```

Tecnicamente correto. Você ainda não sabe *onde*, nem *o que* faltou.

O QUE O GCC DIZ

```
error: expected ')' before ';'
    5 | int y = (x * 2;
      |        ~      ^
      |               )
```

Diz a linha, a coluna, o que esperava, e sugere a correção.

CONSEQUÊNCIA PRÁTICA

Boa parte da qualidade percebida de um compilador está nas mensagens. Vocês vão escrever mensagens de erro já na primeira entrega, e eu vou corrigi-las.

O paradoxo do primeiro compilador

- O compilador de C mais usado do mundo é escrito em C.

O paradoxo do primeiro compilador

- O compilador de C mais usado do mundo é escrito em C.
- Para compilá-lo, você precisa de um compilador de C.

O paradoxo do primeiro compilador

- O compilador de C mais usado do mundo é escrito em C.
- Para compilá-lo, você precisa de um compilador de C.
- Que também é escrito em C.

O paradoxo do primeiro compilador

- O compilador de C mais usado do mundo é escrito em C.
- Para compilá-lo, você precisa de um compilador de C.
- Que também é escrito em C.

COMO SE SAI DISSO

O primeiro foi escrito em **assembly**, à mão. Ele compilou uma versão mínima de C, que compilou uma versão melhor, que compilou a atual. Chama-se *bootstrapping*.

O paradoxo do primeiro compilador

- O compilador de C mais usado do mundo é escrito em C.
- Para compilá-lo, você precisa de um compilador de C.
- Que também é escrito em C.

COMO SE SAI DISSO

O primeiro foi escrito em **assembly**, à mão. Ele compilou uma versão mínima de C, que compilou uma versão melhor, que compilou a atual. Chama-se *bootstrapping*.

Um compilador é só um programa. Você vai escrever um.

6
O mapa

Teoria e prática, lado a lado

Teoria	**Vira**
Autômatos finitos e expressões regulares	análise léxica
Gramáticas livres de contexto e autômatos de pilha	análise sintática
Gramáticas com atributos	análise semântica
Máquinas de Turing	os limites de tudo

Teoria e prática, lado a lado

Teoria	Vira
Autômatos finitos e expressões regulares	análise léxica
Gramáticas livres de contexto e autômatos de pilha	análise sintática
Gramáticas com atributos	análise semântica
Máquinas de Turing	os limites de tudo

Cada linha de teoria aparece imediatamente antes da fase que a consome.

Um analisador léxico consegue verificar se os parênteses de um programa estão balanceados?

A resposta, e o que faremos com ela

- A intuição diz que sim. A resposta é não.

A resposta, e o que faremos com ela

- A intuição diz que sim. A resposta é não.
- E não por limitação de implementação, nem por falta de esperteza.

A resposta, e o que faremos com ela

- A intuição diz que sim. A resposta é não.
- E não por limitação de implementação, nem por falta de esperteza.
- Vamos **demonstrar** que é impossível, e a demonstração cabe em meia página.

A resposta, e o que faremos com ela

- A intuição diz que sim. A resposta é não.
- E não por limitação de implementação, nem por falta de esperteza.
- Vamos **demonstrar** que é impossível, e a demonstração cabe em meia página.

POR QUE ISSO IMPORTA

É essa impossibilidade que obriga todo compilador a ter uma segunda fase de análise. A arquitetura que vimos hoje não é escolha de projeto, é consequência de um teorema.

7

Como atravessar

Como estudar isto

- **Autômato se aprende desenhando.** Ler o autômato do slide não funciona. Refaça no papel, sem olhar.

Como estudar isto

- **Autômato se aprende desenhando.** Ler o autômato do slide não funciona. Refaça no papel, sem olhar.
- **Demonstração se aprende reproduzindo.** Leia, feche o material, escreva de memória. Onde travar é o que você não entendeu.

Como estudar isto

- **Autômato se aprende desenhando.** Ler o autômato do slide não funciona. Refaça no papel, sem olhar.
- **Demonstração se aprende reproduzindo.** Leia, feche o material, escreva de memória. Onde travar é o que você não entendeu.
- **Implemente antes da aula que precisa.** O código escrito na véspera faz a aula seguinte render o dobro.

Como estudar isto

- **Autômato se aprende desenhando.** Ler o autômato do slide não funciona. Refaça no papel, sem olhar.
- **Demonstração se aprende reproduzindo.** Leia, feche o material, escreva de memória. Onde travar é o que você não entendeu.
- **Implemente antes da aula que precisa.** O código escrito na véspera faz a aula seguinte render o dobro.

Esta disciplina pune leitura passiva mais que a média.

Os três modos de fracassar

- **Deixar a atividade online para a semana da prova.** São seis horas de matemática, e elas não comprimem.

Os três modos de fracassar

- **Deixar a atividade online para a semana da prova.** São seis horas de matemática, e elas não comprimem.
- **Começar a primeira entrega na véspera.** Um analisador léxico não se escreve numa noite.

Os três modos de fracassar

- **Deixar a atividade online para a semana da prova.** São seis horas de matemática, e elas não comprimem.
- **Começar a primeira entrega na véspera.** Um analisador léxico não se escreve numa noite.
- **Não desenhar à mão.** Quem só lê chega na metade do curso sem conseguir montar uma gramática.

Os três modos de fracassar

- **Deixar a atividade online para a semana da prova.** São seis horas de matemática, e elas não comprimem.
- **Começar a primeira entrega na véspera.** Um analisador léxico não se escreve numa noite.
- **Não desenhar à mão.** Quem só lê chega na metade do curso sem conseguir montar uma gramática.

Os três são evitáveis, e nenhum deles é sobre inteligência.

O combinado

Avaliação

- **G1** com peso 1, **G2** com peso 2
- Cada um: 6,0 de prova e 4,0 de trabalhos
- **Exame Final** opcional
- Frequência mínima de 75%

O projeto, em quatro etapas

- Léxico à mão *(individual)*
- Sintático e árvore *(em grupo)*
- Análise semântica
- Execução

O combinado

Avaliação

- **G1** com peso 1, **G2** com peso 2
- Cada um: 6,0 de prova e 4,0 de trabalhos
- **Exame Final** opcional
- Frequência mínima de 75%

O projeto, em quatro etapas

- Léxico à mão *(individual)*
- Sintático e árvore *(em grupo)*
- Análise semântica
- Execução

DUAS COISAS QUE QUASE NINGUÉM SABE

O **Exame Final** é direito de qualquer aluno com 75% de frequência que queira subir a média, aprovado ou não. E a **primeira etapa é individual**.

O combinado

Avaliação

- **G1** com peso 1, **G2** com peso 2
- Cada um: 6,0 de prova e 4,0 de trabalhos
- **Exame Final** opcional
- Frequência mínima de 75%

O projeto, em quatro etapas

- Léxico à mão *(individual)*
- Sintático e árvore *(em grupo)*
- Análise semântica
- Execução

DUAS COISAS QUE QUASE NINGUÉM SABE

O **Exame Final** é direito de qualquer aluno com 75% de frequência que queira subir a média, aprovado ou não. E a **primeira etapa é individual**.

Todas as datas estão no plano de ensino, no Canvas.

Antes da próxima aula

- **A primeira atividade online** está liberada no Canvas. São seis horas de fundamentos matemáticos. Comece esta semana.

Antes da próxima aula

- **A primeira atividade online** está liberada no Canvas. São seis horas de fundamentos matemáticos. Comece esta semana.
- **Ambiente funcionando**: Python 3, Java e `antlr4-tools`. O roteiro está no material da aula.

Antes da próxima aula

- **A primeira atividade online** está liberada no Canvas. São seis horas de fundamentos matemáticos. Comece esta semana.
- **Ambiente funcionando**: Python 3, Java e `antlr4-tools`. O roteiro está no material da aula.
- **Papel e caneta.** Sério.

No fim deste curso, você terá escrito um compilador.

E saberá demonstrar por que cada uma de suas partes precisa existir.